Experiment 9

Creating a Token (ERC-20)

Aim

To create an ERC-20 token in Solidity with the Mintable, Burnable and Pausable extensions, deploy it, and exercise its standard functions and events (`transfer`, `approve`, `transferFrom`, `Transfer`, `Approval`).

Theory

A **token** is a smart contract that keeps a ledger of balances (`mapping(address => uint256)`) on top of an existing blockchain, so that a new asset can be created, owned and transferred without a new chain. **ERC-20** is the Ethereum standard interface for *fungible* tokens, where every unit is interchangeable, which lets wallets and exchanges handle any compliant token the same way.

ERC-20 interface

Kind	Members	Purpose
Attributes	`name()`, `symbol()`, `decimals()`, `totalSupply()`	Token metadata and supply (`decimals = 18` means 1 token = 10^18 base units)
Functions	`balanceOf(owner)`, `transfer(to, value)`, `approve(spender, value)`, `allowance(owner, spender)`, `transferFrom(from, to, value)`	Read balances, move tokens, and delegate spending to another address
Events	`Transfer(from, to, value)`, `Approval(owner, spender, value)`	Emitted on every movement / approval so wallets and explorers can index them

ERC-20 vs ERC-721. ERC-20 tokens are *fungible*: balances are plain amounts and any unit equals any other (currencies, voting or utility tokens). ERC-721 tokens are *non-fungible*: each token has a unique `tokenId` and an owner (`ownerOf(tokenId)`), so each one is distinct (collectibles, certificates, property deeds).

Extensions (from OpenZeppelin, all used in this experiment's `VESToken`):

- **Mintable** — an owner-restricted `mint(to, amount)` creates new tokens, so total supply can grow after deployment.
- **Burnable** — `burn(amount)` lets a holder destroy their own tokens, permanently reducing total supply.
- **Pausable** — the owner can `pause()` to halt every transfer (an emergency stop) and `unpause()` to resume.

Tools used: Node.js 24, Hardhat 2.29 with `@nomicfoundation/hardhat-toolbox` (bundles `ethers.js` v6), OpenZeppelin Contracts 5.x, Solidity 0.8.24. Run directly in a terminal against a standalone `npx hardhat node` (port 8549), in place of the Remix IDE LearnETH course, a Metamask wallet and a Sepolia/RSK testnet faucet: the token was deployed to a local chain, not a public testnet.

Output

1. Deploy the token and read its attributes

```
=== 1. Deploy the ERC-20 token ===
VESToken deployed at: 0x5FbDB2315678afecb367f032d93F642f64180aa3
name()        : VES Token
symbol()      : VEST
decimals()    : 18
totalSupply() : 1000.0 VEST
balanceOf(Owner): 1000.0 VEST
```

`VESToken` is deployed with an initial supply of 1000 VEST, all credited to the deployer, and its ERC-20 metadata attributes are read back.

2. `transfer`, `approve` and `transferFrom`

```
=== 2. transfer(): Owner -> Alice 100 VEST ===
Transfer event: from Owner to Alice value 100.0 VEST
balanceOf(Owner): 900.0
balanceOf(Alice): 100.0

=== 3. approve() + transferFrom(): Alice lets Bob spend 40 VEST ===
Approval event: owner Alice spender Bob value 40.0 VEST
```

```
allowance(Alice, Bob): 40.0
Bob calls transferFrom(Alice, Bob, 25)
balanceOf(Alice): 75.0
balanceOf(Bob)  : 25.0
allowance(Alice, Bob) now: 15.0
Bob tries to spend 20 more than the remaining 15 allowance: reverted as expected ->
ERC20InsufficientAllowance("0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC", 15000000000000000000,
20000000000000000000)
```

A direct transfer emits `Transfer`; Alice approves Bob for 40 VEST (emitting `Approval`), Bob spends 25 through `transferFrom`, the allowance falls to 15, and a further 20 VEST attempt is rejected with `ERC20InsufficientAllowance`.

3. Mintable and Burnable

```
=== 4. Mintable: owner mints 500 VEST to Bob ===
totalSupply(): 1500.0
balanceOf(Bob): 525.0
Alice (non-owner) tries to mint: reverted as expected ->
OwnableUnauthorizedAccount("0x70997970C51812dc3A010C7d01b50e0d17dc79C8")

=== 5. Burnable: Bob burns 100 VEST ===
totalSupply(): 1400.0
balanceOf(Bob): 425.0
```

Minting raises total supply from 1000 to 1500 and only the owner may do it; Bob's `burn(100)` lowers total supply from 1500 to 1400.

4. Pausable

```
=== 6. Pausable: owner pauses all transfers ===
paused(): true
Alice tries to transfer while paused: reverted as expected -> EnforcedPause()
paused() after unpause(): false
Alice -> Bob 5 VEST succeeds after unpause; balanceOf(Alice): 70.0

=== 7. Final balances ===
Owner : 900.0 VEST
Alice : 70.0 VEST
Bob   : 430.0 VEST
totalSupply(): 1400.0 VEST
```

While paused, every transfer reverts with `EnforcedPause()`; after `unpause()` transfers work again. The final balances (900 + 70 + 430) sum to the total supply of 1400.

(Full console log saved as `deploy_output.log`, and the Hardhat node's own server-side log as `hardhat_node.log`, both in this folder; the contract is `contracts/VESToken.sol`.)

Conclusion

This experiment created an ERC-20 token, `VESToken (VEST)`, and deployed it to a local Hardhat blockchain in place of a Sepolia/RSK testnet deployment via Remix and Metamask. All the standard behaviours were exercised: balances and metadata, direct transfers with `Transfer` events, the approve-then-`transferFrom` allowance flow with `Approval` events, and its failure when the allowance is exceeded. The three extensions each did what the viva topic describes: minting grew the supply from 1000 to 1500 (owner only), burning shrank it to 1400, and pausing froze all transfers until the owner resumed them. Total supply stayed consistent with the sum of balances throughout (1400 = 900 + 70 + 430).